

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Verão de 2021/2022
Teste Global de Época Normal (28 de Junho de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Uma aplicação *web* recebe um pedido que não pode processar, uma vez que deteta a falta de um parâmetro obrigatório na *body*. O *status code* adequado para responder a esta situação é:
A. 200
B. 400
C. 500
D. 404
 2. [1,25] Segundo as regras definidas no protocolo HTTP, para criar um recurso:
A. O método POST pode sempre ser usado.
B. O método GET pode ser usado mas não é aconselhável.
C. O método PUT pode ser usado quando o URI do recurso a criar é conhecido previamente.
D. O método PUT pode ser usado quando o URI do recurso a criar não é conhecido previamente.
 3. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para TODOS os elementos *div* que inclui a classe *special* é:
A. `document.querySelectorAll("div.special")`
B. `document.querySelectorAll("special.div")`
C. `document.querySelectorAll("div#special")`
D. `document.querySelectorAll("div special")`
 4. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Quantos segundos decorrem a partir da afixação de BEGIN na consola e antes que seja afixado END?
A. 2 ou menos segundos
B. 3 segundos, pelo menos
C. 4 segundos, pelo menos
D. 5 ou mais segundos
- ```
let v = 1
function msg(s) { console.log(s); }
function wait() { if (!v) msg("END")
 else setTimeout(() => { wait() }, 1000)}
msg("BEGIN");
setTimeout(() => { v = 0 }, 3000);
setTimeout(() => { wait() }, 1000);
```
5. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *item*, identificados por URIs com path `/items/_id_`, em que `_id_` é o identificador de cada item. Pretende-se aceitar um pedido HTTP para atualizar um recurso *item*. Qual das seguintes configurações da aplicação *express* (*app*) é adequada de acordo com a especificação HTTP?  
A. `app.get("/item/:id/create", createItemWithId)`  
B. `app.put("/items/:id", createItemWithId)`  
C. `app.post("/items/:id", createItemWithId)`  
D. `app.put("/items/", createItemWithId)`
  6. [1,25] O contexto de uma função é capturado:  
A. Sempre que é definida uma função dentro de outra função  
B. Sempre que é definida uma função dentro de outra função e é acessível do exterior da função onde foi definida  
C. Sempre que a função é usada como função construtora  
D. Sempre que é definida uma função *async* dentro de outra função

**GRUPO 2 [4,5 valores]**

7. [1,5] Considere a aplicação *express* definida na listagem seguinte e o erro dado no servidor quando é submetido o formulário. Justifique a razão do *output* na consola.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const express = require('express') const app = express()  app.get(express.urlencoded())  app.get('/hello', (req, res) =&gt; res.send(`   &lt;html&gt;&lt;body&gt;&lt;form method="post"&gt;     Nickname: &lt;input type="text" id="nick"&gt;     &lt;input type="submit"&gt;   &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; `)) app.post('/hello', (req, res) =&gt; {   console.log(req.body.nick)   resp.end() }) app.listen(3000, () =&gt; console.log('Listening...'))</pre> | <div><div>Nickname: <input type="text" value="master"/> <input type="submit" value="Submit"/></div><div>Na submissão do formulário na consola do servidor aparece o seguinte:<br/><br/>undefined</div></div> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

8. [3] Considere uma aplicação *express* com as rotas definidas na listagem seguinte e a sequência de pedidos que produz as vistas apresentadas: 1) GET /degrees/leic, 2) GET /degrees/leic/course, 3) POST /degrees/leic/course. Indique porque com o código em Solution 1 o *browser* produz o aviso apresentado na última imagem com Solution 2 esse aviso já não ocorre.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const degrees = {leic: ['ipw', 'ave', 'pc']}  app.get('/degrees/:deg', (req, res) =&gt; {   res.render('courses', {     courses: degrees[req.params.deg]   }) }) app.get('/degrees/:deg/course', (req, res) =&gt; res.end(`   &lt;html&gt;&lt;body&gt;&lt;form method="post"&gt;     Course: &lt;input type="text" name="course"&gt;     &lt;input type="submit"&gt;   &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; `)) app.post('/degrees/:deg/course', (req, res) =&gt; {   degrees[req.params.deg].push(req.body.course)   // Solution 1   res.render('courses', {     courses: degrees[req.params.deg]   })   // Solution 2   // res.redirect('/degrees/' + req.params.deg)) })</pre> | <div><div><div>localhost:3000/degrees/leic</div><div><b>Courses:</b><ul style="list-style-type: none"><li>ipw</li><li>ave</li><li>pc</li></ul></div></div><div><div>localhost:3000/degrees/leic/course</div><div>Course: <input type="text" value="daw"/> <input type="submit" value="Submit"/></div></div><div><div>localhost:3000/degrees/leic/course</div><div><b>Courses:</b><ul style="list-style-type: none"><li>ipw</li><li>ave</li><li>pc</li><li>daw</li></ul></div><div><div>Resubmit the form?</div><div>The page you're looking for used information that you entered. Returning to that page might trigger a repetition of any action you took there. Do you want to continue?</div><div><input type="button" value="Continue"/> <input type="button" value="Cancel"/></div></div></div></div> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### GRUPO 3 [2 valores]

9. [2] Considere o seguinte excerto de código em JavaScript:

```
1 function Coll(items) {
2 this.removeItem = items.pop
3 }
4 const coll = new Coll([2,4,6,8,10])
5 console.log(coll.removeItem())
```

- [1] A instrução da linha 5 não apresenta o valor 10, como seria esperado, correspondente à remoção do último elemento do *array* *items*. Justifique.
- [1] Realize as alterações necessárias ao código, de modo a que o método `removeItem` tenha o comportamento esperado, ou seja, remova do *array* *items* o elemento que está na última posição.

### GRUPO 4 [6 valores]

10. [3] Implemente uma função `allSettled()` em JavaScript, que recebe um *array* de *Promise* e retorna uma *Promise* que fica resolvida, quando todas as *promises* ficarem resolvidas. A *promise* resultante tem sempre sucesso, mesmo que uma ou mais *promises* recebida como argumento seja resolvida com erro. O *array* resultante tem em cada posição um objeto com o resultado da *promise* correspondente no *array* recebido como argumento. Esse objeto tem 2 propriedades: `status`, que contém a string `fulfilled` ou `rejected` e `value`, com o valor em caso de sucesso ou o erro. Segue-se um exemplo de utilização da função:

```
allSettled([
 Promise.resolve(33),
 new Promise(resolve => setTimeout(() => resolve(66), 0)),
 99,
 Promise.reject(new Error('an error'))
)
.then(values => console.log(values));
// [
// {status: "fulfilled", value: 33},
// {status: "fulfilled", value: 66},
// {status: "fulfilled", value: 99},
// {status: "rejected", value: Error: an error}
//]
```

11. [3] Considere a aplicação KOMER desenvolvida no trabalho prático. Pretende-se implementar funcionalidade de adicionar a mesma receita a vários grupos de uma única vez. Implemente, no módulo `komer-services`, a função `addReceipeToGroups(userId, receipeId, groupsIds)`, que recebe em `userId` o identificador do utilizador, em `receipeId` o identificador da receita, em `groupIds` os identificadores dos grupos a adicionar a receita e retorna uma *promise* que fica resolvida quando a operação estiver concluída, com o identificador dos grupos onde foi e não foi inserida. A implementação desta função chama várias vezes a função `addReceipe(userId, receipeId, groupId)` do módulo `komer-db`, que recebe em `userId` o identificador do utilizador, em `receipeId` o identificador da receita em `groupId` o identificador do grupo e retorna uma *promise* que fica resolvida com sucesso quando a receita for inserida no grupo ou com erro caso exista algum erro ou a receita já exista no grupo. A falha da inserção num dos grupos não invalida a inserção nos outros. Na implementação desta função, use a função `allSettled()` desenvolvida na questão anterior. **Valorizam-se soluções que façam as adições em paralelo.**

NOTA: Na implementação considere apenas as situações de sucesso, ignorando as validações de `userId` e `groupId`. Assuma também que `addReceipe(userId, receipeId, groupId)` já está implementada.